
CPR E 4890: Computer Networking and Data Communications

Lab Experiment #2 – TCP Socket Programming

(Total Points: 100)

Objective

- To understand the basic concepts of TCP socket programming.
- To learn how to create a TCP socket and establish a connection between a client and a server.
- To learn how to send and receive data using a TCP socket.
- To write a client-server programming application using TCP sockets to implement the `ruptime` UNIX command (https://www.unix.com/man_page/linux/1/ruptime/).

Pre-Lab

- **Read** `SocketProgramming.pdf` and **watch** the mini lecture on socket programming.
- Investigate approaches on how to capture the output from `uptime` (or other command-line programs) within your server program. Include two of these approaches in your lab report. You do not need to implement both but should describe how the techniques work.

Lab Expectations

Work through the lab and let the TA know if you have any questions. **Demonstrate your program to the TA after you have completed it.** After the lab, write up a lab report. Be sure to

- 1) Attend the lab **(5 points)**
- 2) Summarize what you learned in a few paragraphs. **(15 points)**
- 3) Include your **two approaches** for capturing `uptime` output from the pre-lab. **(15 points)**
- 4) Include your **well-commented** code and **demo** your code to the TA.
 - a) correct implementation of the TCP socket. **(40 points)**
 - b) correctly parse the output of the `uptime` command. **(20 points)**
 - c) server should remain open indefinitely after serving each request and wait for further connection requests. **(5 points)**

Demonstration Policy

This lab will require you to **demo** your code to the TA. Review the lab manual carefully to ensure you have completed all demonstration requirements. Labs may be demonstrated during:

- **this lab section;**
- **the TA office hours;**
- **or the first hour of the next lab (Lab #3).**

NOTE: Lab reports are due on the day **before** the next lab.

Problem Description

From the `ruptime` man page on a UNIX system:

`ruptime` gives a status line like `uptime` for each machine on the local network; these are formed from packets broadcast by each host on the network once a minute.

In this lab experiment, you are required to implement a simple version of the `ruptime` UNIX command by **writing two programs: a client and a server.**

- You are required to execute and demonstrate the server program (called `ruptimeServer`) on one of the Coover 2061 machines by typing

```
$ ./ruptimeServer
```

- You are required to execute and demonstrate the client (called `ruptimeClient`) from one of the Coover 2061 machines (you may use the machine that your `ruptimeServer` is running on) and will connect to the copy of your server running on this machine. Use the command:

```
$ ./ruptimeClient 192.168.254.X
```

Note that X is dependent upon the machine. Since this lab will be performed individually, you may use 127.0.0.1 to execute your programs on the localhost interface.

- When the server is contacted by the client, the server will execute the UNIX shell command `uptime` and send the system uptime to the client.
- The client will display a server's address followed by the uptime information received from that server. For example, if one of the machines contacted was 192.168.254.2, the output from `ruptimeClient` for that machine could be:

```
192.168.254.2: 10:47am up 27 hour(s), 50 min(s), 1 user(s), load average: 0.18, 0.26, 0.20
```

- After printing the uptime information for the server, the client then quits. **The server remains open and waits for further connection requests.**

Procedure

- Write the two programs **`ruptimeServer.c`** and **`ruptimeClient.c`** in C under Linux using your favorite text editor (VS Code, pico, vi, emacs, etc.) Make sure the code is well-commented, and don't forget to do error-checking.
 - "Well-commented": e.g., include function headers describing parameters and return values, explain complex logic, and comment key socket operations.
 - "Error-checking": e.g., check return values from `socket()`, `bind()`, `listen()`, `accept()`, `send()`, `recv()`, and handle connection failures gracefully depending on your code structure.
- The two programs communicate using TCP sockets.
- Compile your programs using `gcc`.
 - Example usage: `gcc -o file file.c`, where `file.c` is your code, and `file` is the required executable file. Note that you can link to other libraries as needed, such as the math library using `-lm`.
 - You may run your program by typing the full path to your compiled executable. For example: `./file arguments`
- The `ruptimeServer` program should listen on a TCP port with a number chosen between 1024 and 65535. This parameter can be passed to the server program through an argument if you wish.
 - If the port you choose is already in use, select a different port number. You can check if a port is in use with commands like `netstat` or `lsof`.
- Test your programs by running `ruptimeClient` and inspecting the output. Repeat several times to make sure that your programs are working properly.
- **Demonstrate your programs to the lab TA.**
- **Submit a copy of `ruptimeServer.c` and `ruptimeClient.c` with your lab report.**

Notes

- The localhost has the IP address of 127.0.0.1.
- At a minimum, your server should display uptime with hours, minutes, and seconds.
- If there is any missing information, you may make any reasonable assumptions, but clearly state these assumptions in your solution.
- You do not have to implement any of the command line options for the `ruptime` client program.
- **Using the 489labuser account is *not* necessary for this lab.**

Relevant Manual Pages and Resources

- `socket(2)` <https://man7.org/linux/man-pages/man2/socket.2.html>
- `sockaddr(3type)` <https://man7.org/linux/man-pages/man3/sockaddr.3type.html>
- `ip(7)` <https://man7.org/linux/man-pages/man7/ip.7.html>
- `bind(2)` <https://man7.org/linux/man-pages/man2/bind.2.html>
- `listen(2)` <https://man7.org/linux/man-pages/man2/listen.2.html>
- `accept(2)` <https://man7.org/linux/man-pages/man2/accept.2.html>
- `send(2)` <https://man7.org/linux/man-pages/man2/send.2.html>
- `recv(2)` <https://man7.org/linux/man-pages/man2/recv.2.html>
- `close(2)` <https://man7.org/linux/man-pages/man2/close.2.html>
- `perror(3)` <https://man7.org/linux/man-pages/man3/perror.3.html>

Perhaps the most comprehensive, commonly referenced, and openly available guide for network programming (including using UNIX sockets) is Brian Jorgensen's guide, available here:

- Brian Jorgensen's Socket Programming Guide - <https://beej.us/guide/bgnet/html/split/>